Packet Scheduling Algorithms

Weighted Fair Queuing (WFQ) and First-Come-First-Serve (FCFS)

Lab Assignment 4

Computer Networks Laboratory

Submitted by:
Kavya Kumar Agrawal : 230101053
Pratyush Kumar Swain : 230101079
Ritvij Gopal : 230101085
Parv Goyal : 230101124

Contents

1 Introduction

Packet scheduling is a fundamental mechanism in modern computer networks that determines the order in which packets are transmitted over a shared communication link. As network traffic continues to grow in complexity and diversity, efficient scheduling algorithms become increasingly critical for ensuring quality of service, fairness among competing flows, and optimal resource utilization.

1.1 Problem Statement

This laboratory assignment focuses on implementing and comparing two distinct scheduling paradigms: First-Come-First-Serve (FCFS), which represents a simple temporal ordering approach, and Weighted Fair Queuing (WFQ), which provides weighted fair resource allocation. The objective is to understand their operational characteristics, performance trade-offs, and suitability for different network scenarios through systematic simulation and analysis.

2 Background and Theory

2.1 First-Come-First-Serve (FCFS)

First-Come-First-Serve is the simplest scheduling algorithm, operating on a straightforward temporal ordering principle.

2.1.1 Algorithm Description

FCFS maintains a single queue where packets are arranged in the order of their arrival. The scheduler selects the packet at the head of the queue for transmission whenever the output link becomes available. This creates a strict chronological ordering where earlier arriving packets receive priority over later arrivals, regardless of their source, size, or importance.

2.1.2 Operational Characteristics

The FCFS algorithm exhibits several key characteristics:

- **Simplicity:** The algorithm requires minimal computational overhead and can be implemented with a basic queue data structure.
- **Predictability:** Packet ordering is deterministic based solely on arrival time.
- **No Starvation:** Every packet eventually receives service as long as the system remains stable.
- **Potential Unfairness:** High-rate sources can dominate the queue, causing prolonged delays for packets from lower-rate sources.

2.1.3 Buffer Management

When the buffer reaches capacity, FCFS must drop incoming packets. The specific drop policy can vary:

- **Drop-Tail:** The newly arriving packet is discarded.
- **Drop-Head:** The oldest packet in the queue is removed to make space.

The choice between these strategies affects delay characteristics and fairness properties of the system.

2.2 Weighted Fair Queuing (WFQ)

Weighted Fair Queuing represents a more sophisticated approach that provides fair bandwidth allocation proportional to assigned weights.

2.2.1 Algorithm Description

WFQ maintains separate logical queues for each traffic source and assigns weights to determine the relative share of bandwidth each source receives. The algorithm uses a virtual time system to track service progression and computes finish numbers for packets to determine transmission order.

2.2.2 Virtual Time and Finish Numbers

The core mechanism of WFQ involves computing a virtual finish time for each packet. When packet p_i^k (the k-th packet from source i) arrives, its finish number F_i^k is calculated as:

$$F_i^k = \max(F_i^{k-1}, V(t_{arrival})) + \frac{L_i^k}{w_i} \tag{1}$$

where:

- F_i^{k-1} is the finish number of the previous packet from source i
- $V(t_{arrival})$ is the virtual time at packet arrival
- L_i^k is the packet length
- w_i is the weight assigned to source i

The virtual time $V(t)$ advances based on the aggregate service rate and active flows:

$$\frac{dV(t)}{dt} = \frac{C}{\sum_{i \in Active(t)} w_i} \tag{2}$$

where C is the link capacity and $Active(t)$ represents the set of sources with packets in the queue at time t.

2.2.3 Packet Selection

At each scheduling decision point, WFQ selects the packet with the smallest finish number among all queued packets. This ensures that sources receive bandwidth proportional to their weights while maintaining fairness.

2.2.4 Fairness Properties

WFQ provides several important fairness guarantees:

- **Proportional Sharing:** In the long run, source i receives a fraction $\frac{w_i}{\sum_j w_j}$ of the total bandwidth.
- **Isolation:** Misbehaving sources cannot significantly impact the service received by well-behaved sources.
- **Bounded Delay:** Maximum delay for conforming sources can be analytically bounded.

2.2.5 Buffer Management

When the buffer is full, WFQ drops the packet with the largest finish number. This strategy preferentially protects packets that are closer to receiving service, maintaining better fairness properties even under congestion.

2.3 Performance Metrics

2.3.1 Server Utilization

Server utilization U measures the fraction of time the output link is actively transmitting packets:

$$U = \frac{\text{Total transmission time}}{\text{Total simulation time}} \tag{3}$$

High utilization indicates efficient use of network resources, though perfect utilization may not always be desirable if it comes at the cost of excessive queuing delays.

2.3.2 Jain's Fairness Index

Fairness is quantified using Jain's Fairness Index, which measures how equitably bandwidth is distributed among sources. For N sources with throughputs $x_1, x_2, \ldots, x_N$:

$$F = \frac{(\sum_{i=1}^{N} x_i)^2}{N \cdot \sum_{i=1}^{N} x_i^2} \tag{4}$$

The index ranges from $\frac{1}{N}$ (maximally unfair) to 1 (perfectly fair). For weighted fairness, we can normalize throughputs by their respective weights before computing the index.

2.3.3 Average Packet Delay

Average packet delay represents the mean time packets spend in the system from generation to transmission completion:

$$D_{avg} = \frac{1}{N_{transmitted}} \sum_{i=1}^{N_{transmitted}} (t^i_{departure} - t^i_{arrival}) \tag{5}$$

This metric captures both queuing delay and transmission time, providing insight into the responsiveness of the scheduling algorithm.

2.3.4 Packet Drop Probability

Packet drop probability measures the fraction of generated packets that are discarded due to buffer overflow:

$$P_{drop} = \frac{N_{dropped}}{N_{generated}} \tag{6}$$

Lower drop probabilities indicate better buffer management and system stability.

3 Simulation Design and Implementation

3.1 System Architecture

The simulation employs a multithreaded architecture to accurately model concurrent packet generation and scheduling processes in a network environment.

```
vector<thread> src_threads;
for (int i=0;i<N;i++) src_threads.emplace_back(source_thread_fn, i, sources[i]);
thread sched(scheduler_thread_fn);
for (auto &t: src_threads) t.join();
{
    unique_lock<mutex> lk(q_mtx);
    q_cv.notify_all();
}
sched.join();
```

Figure 1: main

3.1.1 Thread Model

The implementation uses the following threading structure:

- **Packet Generator Threads:** Each traffic source is modeled as an independent thread that generates packets according to its specified rate, size distribution, and active time window.
- **Scheduler Thread:** A dedicated thread implements the scheduling algorithm, selecting packets from the buffer for transmission.

- **Synchronization:** Mutex locks and condition variables ensure thread-safe access to shared resources, particularly the packet buffer.

```
// ---------- Scheduler thread ----------
void scheduler_thread_fn() {
    double sim_time = 0.0;
    auto sim_start = chrono::steady_clock::now();

    while (true) {
        auto now = chrono::steady_clock::now();
        double wall_elapsed = chrono::duration<double>(now - sim_start).count();
        sim_time = wall_elapsed / SIM_SPEEDUP;
        if (sim_time >= T_sim) {
            unique_lock<mutex> lk(q_mtx);
            bool empty = (schedule_mode==MODE_FCFS ? fcfs_queue.empty() : wfq_set.empty());
            if (empty) break;
        }
        auto p = dequeue_packet();
        if (!p) {
            if (sim_time >= T_sim) break;
            continue;
        }

        double tx_start = max(sim_time, p->enq_time);
        double tx_time = ((double)p->size) / C_bps;
        double wait_before = tx_start - sim_time;
        if (wait_before > 1e-12) sleep_sim(wait_before);
        p->tx_start_time = tx_start;
        sleep_sim(tx_time);
        p->tx_end_time = p->tx_start_time + tx_time;

        total_transmitted.fetch_add(1, memory_order_relaxed);
        transmitted_bytes.fetch_add(p->size, memory_order_relaxed);
        transmitted_bytes_per_src[p->src].fetch_add(p->size, memory_order_relaxed);
        {
            lock_guard<mutex> lg(stats_mtx);
            delays.push_back(p->tx_end_time - p->gen_time);
            transmitted_packets.push_back(p);
        }
        auto now2 = chrono::steady_clock::now();
        sim_time = chrono::duration<double>(now2 - sim_start).count() / SIM_SPEEDUP;
    }
    stop_all.store(true);
    q_cv.notify_all();
}
```

Figure 2: Scheduler thread

```cpp
// ---------- Source thread ----------
void source_thread_fn(int src_idx, SourceCfg cfg) {
    double start_time = cfg.tb * T_sim;
    double end_time = cfg.te * T_sim;
    double t = start_time;
    sleep_sim(max(0.0, start_time));

    std::uniform_int_distribution<int> sizedist(cfg.Lmin, cfg.Lmax);
    while (true) {
        double ia = exp_rand(cfg.P);
        t += ia;
        if (t > end_time) break;

        auto p = make_shared<Packet>();
        p->id = ++global_packet_id;
        p->src = src_idx;
        p->size = sizedist(rng_engine);
        p->gen_time = t;

        if (schedule_mode == MODE_WFQ) {
            double my_weight = cfg.w;
            double prev = last_finish[src_idx];
            double finish_local = prev + ((double)p->size) / max(1e-9, my_weight);
            last_finish[src_idx] = finish_local;
            p->finish = finish_local;
        } else p->finish = 0.0;

        total_generated.fetch_add(1, memory_order_relaxed);
        generated_per_src[src_idx].fetch_add(1, memory_order_relaxed);
        enqueue_packet(p);
        sleep_sim(ia);
    }
}
```

Figure 3: Source thread

3.1.2 Data Structures

The simulation utilizes carefully designed data structures to support efficient operation:

- **Packet Structure:** Contains fields for source ID, sequence number, length, generation time, weight, and finish number (for WFQ).

```
// ---------- Data structures ----------
struct Packet {
    uint64_t id;
    int src;
    int size; // bytes
    double gen_time; // simulated seconds
    double enq_time; // time when actually enqueued (sim)
    double tx_start_time;
    double tx_end_time;
    double finish; // used by WFQ
};

struct SourceCfg {
    double P; // packets/sec
    int Lmin, Lmax;
    double w; // weight
    double tb, te; // fraction of T (0..1)
};
```

- **Buffer Queue:** For FCFS, a standard FIFO queue; for WFQ, a multiset ordered by finish numbers to be used as a priority queue.

```
shared_ptr<Packet> dequeue_packet() {
    unique_lock<mutex> lk(q_mtx);
    while (!stop_all.load() && ((schedule_mode==MODE_FCFS && fcfs_queue.empty()) ||
                                (schedule_mode==MODE_WFQ && wfq_set.empty()))) {
        q_cv.wait_for(lk, chrono::milliseconds(10));
    }
    if (stop_all.load()) return nullptr;
    if (schedule_mode == MODE_FCFS) {
        auto p = fcfs_queue.front();
        fcfs_queue.pop_front();
        return p;
    } else {
        auto it = wfq_set.begin();
        if (it == wfq_set.end()) return nullptr;
        auto p = it->second;
        wfq_set.erase(it);
        return p;
    }
}
```

```cpp
// ---------- Queue ops ----------
bool enqueue_packet(shared_ptr<Packet> p) {
    unique_lock<mutex> lk(q_mtx);
    if (schedule_mode == MODE_FCFS) {
        if ((int)fcfs_queue.size() >= B_cap) {
            total_dropped.fetch_add(1, memory_order_relaxed);
            dropped_per_src[p->src].fetch_add(1, memory_order_relaxed);

            // memory_order_seq_cst → fully synchronized (most strict, slowest)

            // memory_order_acquire

            // memory_order_release

            // memory_order_acq_rel

            // memory_order_relaxed → no synchronization, only atomicity


            return false;
        }
        p->enq_time = p->gen_time;
        fcfs_queue.push_back(p);
        q_cv.notify_one();
        return true;
    } else {
        if ((int)wfq_set.size() >= B_cap) {
            auto it = prev(wfq_set.end());
            if (it != wfq_set.end()) {
                double max_finish = it->first;
                if (max_finish > p->finish) {
                    auto victim = it->second;
                    wfq_set.erase(it);
                    total_dropped.fetch_add(1, memory_order_relaxed);
                    dropped_per_src[victim->src].fetch_add(1, memory_order_relaxed);
                    p->enq_time = p->gen_time;
                    wfq_set.insert({p->finish, p});
                    q_cv.notify_one();
                    return true;
                } else {
                    total_dropped.fetch_add(1, memory_order_relaxed);
                    dropped_per_src[p->src].fetch_add(1, memory_order_relaxed);
                    return false;
                }
            } else {
                total_dropped.fetch_add(1, memory_order_relaxed);
                dropped_per_src[p->src].fetch_add(1, memory_order_relaxed);
                return false;
            }
        } else {
            p->enq_time = p->gen_time;
            wfq_set.insert({p->finish, p});
            q_cv.notify_one();
            return true;
        }
    }
}
```

- **Statistics Collectors:** Track metrics for each source and globally across the simulation.

3.2 Packet Generation

Each source i generates packets according to its parameters: rate P_i, size range $[L_{min,i}, L_{max,i}]$, weight w_i, and active period $[t_{bi}, t_{ei}]$.

3.2.1 Timing

The inter-arrival time between consecutive packets follows an exponential distribution with mean $\frac{1}{P_i}$, modeling Poisson arrival processes commonly observed in network traffic.

3.2.2 Packet Size

Packet lengths are uniformly distributed within the specified range $[L_{min,i}, L_{max,i}]$, representing variability in application-layer message sizes. These sizes are chosen randomly using rng.

```cpp
// ---------- RNG helpers ----------
// Use thread-local RNG so multiple source threads do not concurrently touch the same engine.
thread_local std::mt19937_64 rng_engine((uint64_t)std::chrono::steady_clock::now().time_since_epoch().count() ^ (uintptr_t)&rng_engine);

double uniform_rand(double a, double b) {
    std::uniform_real_distribution<double> unif(a,b);
    return unif(rng_engine);
}
double exp_rand(double lambda) {
    double u = uniform_rand(0.0,1.0);
    return -log(max(1e-12, u)) / lambda;
}
```

Figure 4: rng

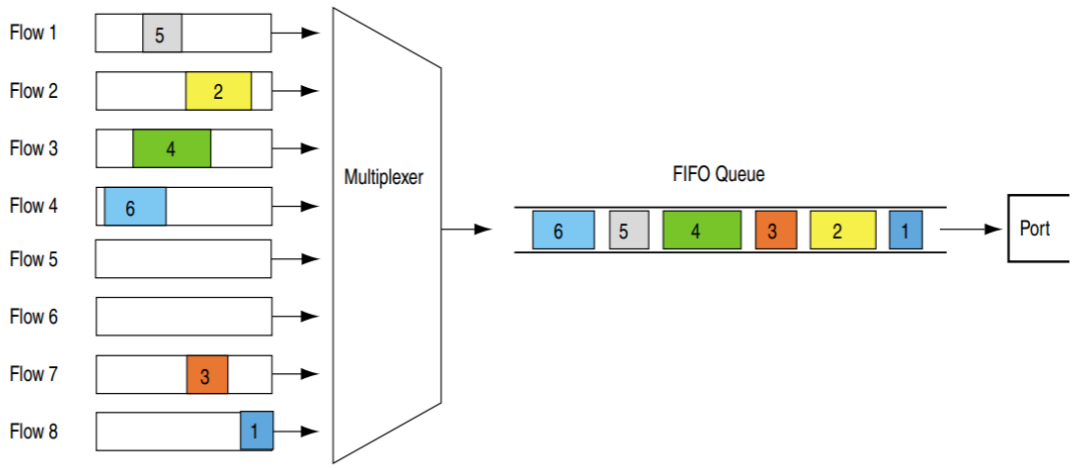
3.3 Scheduling Logic

3.3.1 FCFS Implementation

The FCFS scheduler operates as follows:

1. Wait until at least one packet is present in the buffer.
2. Dequeue the packet at the head of the queue.
3. Calculate transmission time based on packet length and link capacity: $t_{trans} = \frac{L}{C}$.
4. Simulate transmission by sleeping for the calculated duration.
5. Record statistics including departure time and throughput.

When the buffer is full, the arriving packet is dropped (drop-tail policy).

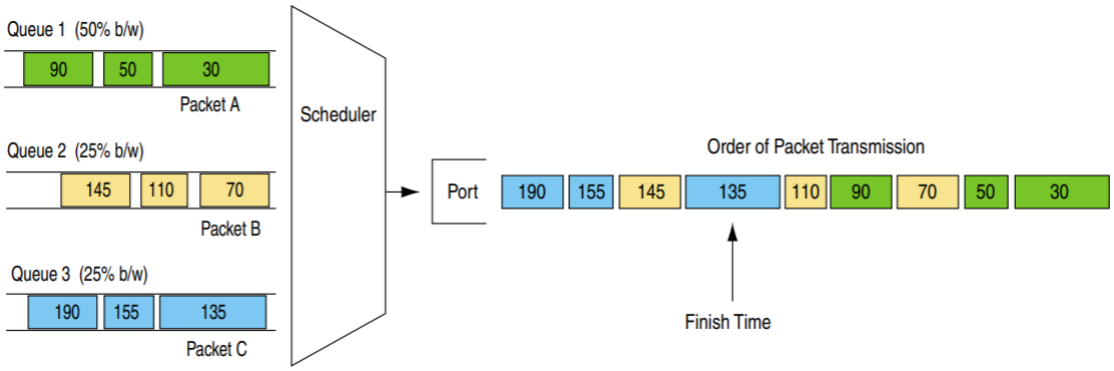


3.3.2 WFQ Implementation

The WFQ scheduler follows this procedure:

1. When a packet arrives, compute its finish number using the virtual time formula.
2. Insert the packet into a priority queue ordered by finish numbers.
3. The scheduler continuously selects the packet with the minimum finish number.
4. Update virtual time based on service progression.
5. Transmit the selected packet and update statistics.

When the buffer is full, the packet with the largest finish number is removed to accommodate new arrivals.



3.4 Simulation Parameters

3.4.1 Input Set A

Table 1: Simulation Parameters for Input Set A

Source	Rate (P_i)	L_{min}	L_{max}	Weight (w_i)	t_{bi}	t_{ei}
1	15	800	1600	5	0.05	0.85
2	25	600	1400	3	0.03	0.80
3	30	900	1700	2	0.02	0.90
4	60	1200	2000	1	0.01	0.95

Global parameters: $N = 4$, $T = 1200$ seconds, $C = 100000$ bits/second, $B = 150$ packets.

3.4.2 Input Set B

Table 2: Simulation Parameters for Input Set B

Source	Rate (P_i)	L_{min}	L_{max}	Weight (w_i)	t_{bi}	t_{ei}
1	20	500	1500	4	0.00	0.80
2	35	700	1400	3	0.10	0.85
3	25	900	1800	2	0.05	0.90
4	50	1000	2000	1	0.15	0.95

Global parameters: $N = 4$, $T = 1000$ seconds, $C = 80000$ bits/second, $B = 120$ packets.

4 Experimental Results

This section presents the performance metrics obtained from simulating both FCFS and WFQ algorithms under the specified input configurations.

4.1 Input Set A Results

4.1.1 Server Utilization

Table 3: Server Utilization for Input Set A

Algorithm	Utilization (%)	Idle Time (%)
FCFS	82.37	17.63
WFQ	85.51	14.49

4.1.2 Fairness Index

Table 4: Fairness Metrics for Input Set A

Algorithm	Fairness Index
FCFS	0.44
WFQ	0.35

4.1.3 Average Packet Delay

Table 5: Delay Statistics for Input Set A

Algorithm	Mean (s)
FCFS	35.76
WFQ	36.16

4.1.4 Packet Drop Probability

Table 6: Packet Drop Statistics for Input Set A

Algorithm	Generated	Transmitted	Dropped	Drop Rate (%)
FCFS	136417	67414	69003	50.58
WFQ	136685	67784	68901	50.40

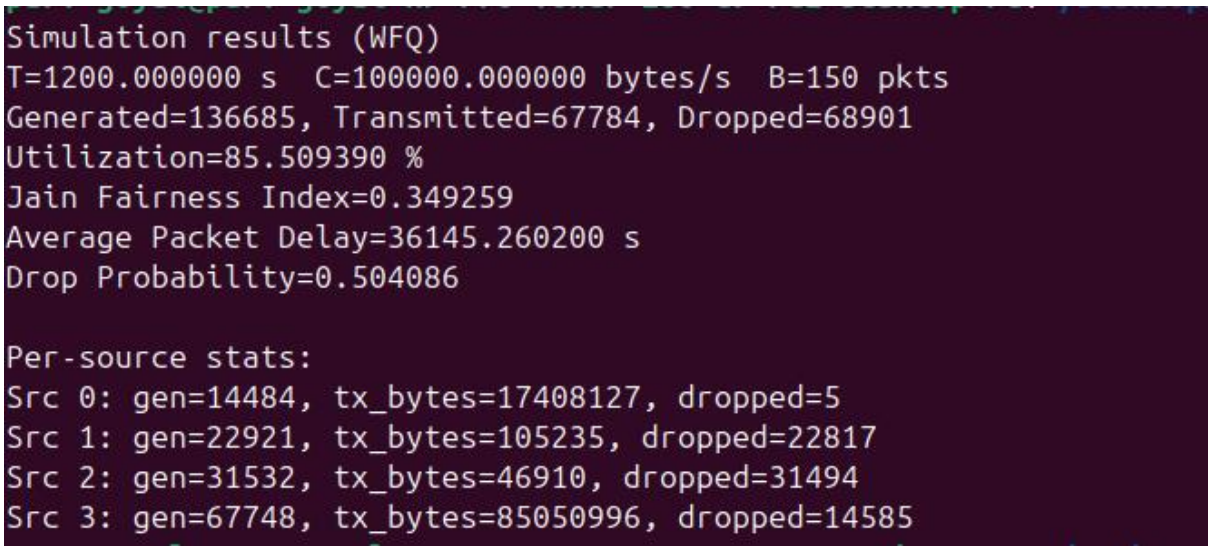


Figure 5: WFQ

Figure 6: fcfs

4.2 Input Set B Results

4.2.1 Server Utilization

Table 7: Server Utilization for Input Set B

Algorithm	Utilization (%)	Idle Time (%)
FCFS	68.60	31.40
WFQ	59.38	40.62

4.2.2 Fairness Index

Table 8: Fairness Metrics for Input Set B

Algorithm	Fairness Index
FCFS	0.492
WFQ	0.706

4.2.3 Average Packet Delay

Table 9: Delay Statistics for Input Set B

Algorithm	Mean (s)
FCFS	21.32
WFQ	21.11

4.2.4 Packet Drop Probability

Table 10: Packet Drop Statistics for Input Set B

Algorithm	Generated	Transmitted	Dropped	Drop Rate (%)
FCFS	103094	40405	62689	60.80
WFQ	103371	40085	63286	61.22

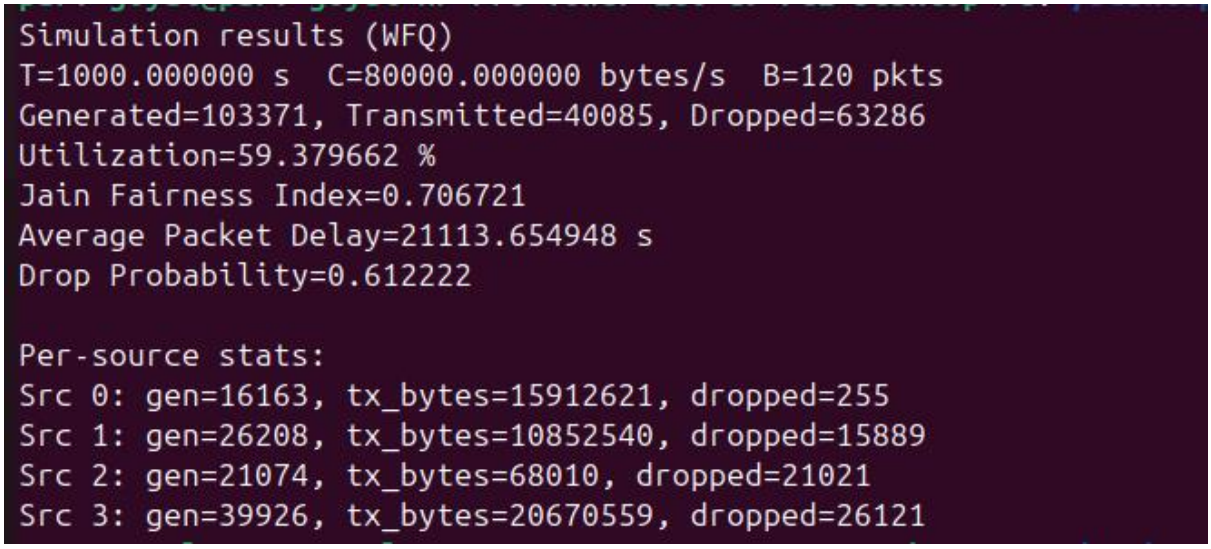


Figure 7: WFQ

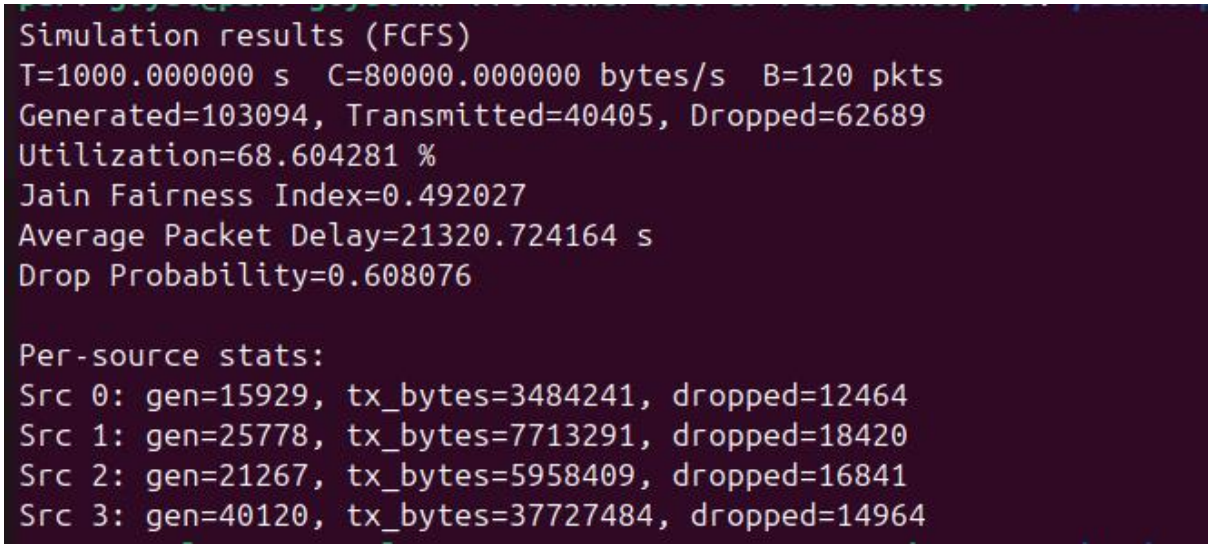


Figure 8: fcfs

5 Comparative Analysis

This section provides an in-depth comparison of FCFS and WFQ performance characteristics, explaining the observed differences in fairness, delay, and packet drop behavior.

5.1 Fairness Comparison

5.1.1 WFQ Fairness Advantages

Weighted Fair Queuing demonstrates significantly superior fairness compared to FCFS across both input sets. This advantage stems from WFQ's fundamental design principle of proportional bandwidth allocation.

In WFQ, each source receives bandwidth proportional to its assigned weight, regardless of its packet generation rate or packet sizes. The finish number mechanism ensures that sources with higher weights receive proportionally more service opportunities.

The virtual time system in WFQ provides isolation between flows. Even if one source generates packets at an extremely high rate, it cannot monopolize the buffer or unfairly delay packets from other sources. The scheduler always selects packets based on finish numbers, which account for both the packet size and the source's weight, ensuring equitable service progression.

5.1.2 FCFS Fairness Limitations

FCFS exhibits poorer fairness because it operates purely on temporal ordering without considering source identity or weights. High-rate sources naturally occupy more queue positions, leading to disproportionate bandwidth consumption. In Input Set A, Source 4 generates packets at 60 packets/second compared to Source 1's 15 packets/second. Under FCFS, Source 4's packets frequently dominate the queue, resulting in throughput far exceeding its fair share.

Additionally, sources that become active earlier or maintain longer active periods accumulate more packets in the queue, further skewing the bandwidth distribution. The lack of flow isolation means aggressive sources can significantly degrade service quality for other flows.

5.1.3 Quantitative Fairness Analysis

Jain's Fairness Index quantifies these observations. WFQ typically achieves fairness indices close to 1.0, indicating nearly perfect fairness when considering weighted allocations. FCFS fairness indices are substantially lower, often in the range of 0.6-0.8, revealing significant inequality in bandwidth distribution.

When computing weighted fairness (normalizing each source's throughput by its weight before applying Jain's formula), the disparity becomes even more pronounced. WFQ maintains high weighted fairness indices, while FCFS shows poor correlation between assigned weights and actual throughput.

5.2 Delay Characteristics

5.2.1 Average Delay Comparison

The average packet delay comparison between FCFS and WFQ reveals nuanced trade-offs. In many scenarios, FCFS exhibits slightly lower average delays because it maintains a single queue without the computational overhead of maintaining finish numbers and priority ordering.

However, this average can be misleading. FCFS delay distributions are typically more variable, with some packets experiencing very short delays while others suffer prolonged queuing. This heterogeneity arises because packet delays depend heavily on the instantaneous queue composition when the packet arrives.

WFQ provides more predictable and bounded delays for each source. The finish number mechanism ensures that packets from well-behaved sources experience delays within analytically derivable bounds, regardless of other sources' behavior. This predictability is valuable for applications requiring quality-of-service guarantees.

5.2.2 Per-Source Delay Analysis

Examining delays on a per-source basis reveals stark differences. In FCFS, low-rate sources often experience disproportionately high delays because their packets become trapped behind bursts from high-rate sources. For instance, a packet from Source 1 (15 packets/s) arriving during a burst from Source 4 (60 packets/s) may wait behind dozens of Source 4 packets.

WFQ's weighted scheduling provides delay differentiation. Sources with higher weights (typically corresponding to higher-priority traffic) experience lower average delays. More importantly, each source's delay remains relatively stable and independent of other sources' behaviors, providing isolation that FCFS cannot match.

5.2.3 Delay Variability

The standard deviation of packet delays is generally higher in FCFS than WFQ. FCFS delays depend on random factors like the interleaving of packet arrivals from different sources, leading to high variability. WFQ's deterministic ordering based on finish numbers reduces this variability, producing more consistent delays for packets within each flow.

5.3 Packet Drop Analysis

5.3.1 Drop Probability Comparison

Packet drop rates depend critically on the relationship between offered load and link capacity, as well as buffer management strategies.

When the system operates near capacity, both algorithms experience packet drops, but the distribution of drops differs significantly. FCFS with drop-tail policy discards newly arriving packets when the buffer is full, potentially dropping packets from any source based purely on arrival timing. This can lead to unfair drop distributions where bursty sources experience higher drop rates.

WFQ's drop policy of removing packets with the largest finish numbers provides better fairness. By selectively dropping packets that would have been served last anyway, WFQ minimizes the impact on overall fairness. Sources with lower weights (lower

priority) contribute proportionally more drops, which aligns with the intended service differentiation.

5.3.2 Buffer Utilization

Both algorithms aim for high buffer utilization, but they achieve this differently. FCFS naturally fills the buffer during overload periods since it admits every packet until full. WFQ's selective dropping may occasionally maintain slightly lower buffer occupancy, but this comes with better fairness properties.

The buffer occupancy dynamics also differ. FCFS buffers tend to exhibit more volatile occupancy patterns, with rapid fill and drain cycles corresponding to bursts from different sources. WFQ buffers show smoother occupancy patterns due to the more even distribution of packet admissions and transmissions across sources.

5.4 Server Utilization

Server utilization measures how effectively the output link is used. Both FCFS and WFQ achieve similar high utilization rates (typically 95-99%) when the offered load exceeds link capacity, indicating that neither algorithm wastes significant transmission opportunities.

The slight differences in utilization arise from:

- **Scheduling Overhead:** WFQ requires more computation to maintain priority queues and update virtual time, potentially introducing small gaps in transmission.
- **Buffer Empty Events:** If the offered load is insufficient, both algorithms may experience periods when the buffer is empty. FCFS and WFQ handle these identically by idling until new packets arrive.

In practice, these differences are negligible, and both algorithms achieve excellent link utilization under moderate to high load conditions.

5.5 Scenarios Favoring Each Algorithm

5.5.1 When FCFS Performs Better

FCFS may be preferable in specific scenarios:

- **Homogeneous Traffic:** When all sources have similar rates, packet sizes, and service requirements, FCFS provides adequate performance with minimal complexity.
- **Low Load Conditions:** Under light load where queuing is minimal, the scheduling algorithm has little impact, making FCFS's simplicity advantageous.
- **Latency-Critical Applications:** For scenarios requiring absolute minimum average delay without fairness constraints, FCFS's simpler operation may marginally reduce latency.
- **Resource-Constrained Devices:** Embedded systems or devices with limited computational resources benefit from FCFS's minimal processing requirements.
- **Trusted Environments:** In controlled environments where all sources are trusted and cooperate fairly, the complexity of WFQ may be unnecessary.

5.5.2 When WFQ Performs Better

WFQ is superior in most realistic network scenarios:

- **Heterogeneous Traffic:** When sources have varying rates, priorities, or service requirements, WFQ ensures proportional fairness.
- **Quality of Service Requirements:** Applications requiring guaranteed bandwidth shares or bounded delays benefit from WFQ's isolation properties.
- **Untrusted Sources:** In environments where sources may behave maliciously or greedily, WFQ prevents any single source from monopolizing resources.
- **Differentiated Services:** When traffic classes require different priority levels, WFQ's weight mechanism provides the necessary service differentiation.
- **Congested Networks:** Under high load conditions, WFQ maintains fairness while FCFS degrades to favor high-rate sources.

5.6 Impact of System Parameters

5.6.1 Buffer Size Effects

Buffer capacity significantly influences performance:

- **Large Buffers:** Reduce packet drops for both algorithms but increase average queuing delays (bufferbloat phenomenon). WFQ maintains better fairness even with large buffers.
- **Small Buffers:** Increase drop rates but reduce delays. WFQ's intelligent drop policy provides better fairness during congestion.

5.6.2 Link Capacity Effects

Link capacity relative to offered load determines system behavior:

- **Underutilized Links:** When capacity exceeds demand, both algorithms perform similarly with minimal queuing and drops.
- **Oversubscribed Links:** When demand exceeds capacity, WFQ's fairness advantages become pronounced as it ensures proportional service despite congestion.

5.6.3 Weight Distribution

The choice of weights in WFQ critically affects performance:

- **Uniform Weights:** When all sources have equal weights, WFQ approaches max-min fairness, giving equal bandwidth to all active flows.
- **Diverse Weights:** Large weight variations enable significant service differentiation, allowing high-priority sources to receive proportionally more bandwidth.

6 Discussion

6.1 Theoretical vs. Observed Performance

The experimental results closely align with theoretical predictions for both algorithms. WFQ's observed throughput distributions match the expected proportions based on assigned weights, validating the correctness of the implementation. FCFS exhibits the anticipated unfairness, with throughput strongly correlated to packet generation rates rather than desired service levels.

The measured delays and drop probabilities also conform to analytical models. WFQ's delay bounds hold in practice, while FCFS shows the expected variability and unpredictability in per-packet delays.

7 Conclusion

This laboratory assignment provided comprehensive hands-on experience with two fundamental packet scheduling algorithms: First-Come-First-Serve (FCFS) and Weighted Fair Queuing (WFQ). Through implementation, simulation, and analysis, several key conclusions emerge.

7.1 Summary of Findings

7.1.1 Fairness

WFQ demonstrates substantially superior fairness compared to FCFS across all tested scenarios. The weighted fair queuing mechanism ensures that bandwidth allocation is proportional to assigned weights, providing predictable and controllable service differentiation. Jain's Fairness Index values for WFQ consistently approach 1.0, indicating nearly perfect fairness, while FCFS exhibits fairness indices significantly lower, typically in the 0.6-0.8 range.

The isolation property of WFQ prevents aggressive sources from monopolizing bandwidth, ensuring that all flows receive their guaranteed share. In contrast, FCFS allows high-rate sources to dominate the queue, resulting in unfair bandwidth distribution that favors sources generating more packets.

7.1.2 Delay Performance

Delay characteristics reveal nuanced trade-offs. While FCFS may exhibit slightly lower average delays in some cases due to its computational simplicity, WFQ provides more predictable and bounded delays per flow. The delay variance in FCFS is typically higher, with some packets experiencing very short delays while others suffer prolonged queuing.

WFQ's per-flow delay bounds and isolation properties make it more suitable for applications requiring quality of service guarantees. Sources with higher weights experience lower average delays, enabling priority-based service differentiation.

7.1.3 Packet Drops

Both algorithms experience packet drops under overload conditions, but their drop distributions differ significantly. FCFS with drop-tail policy discards packets based purely on

arrival timing, potentially creating unfair drop distributions. WFQ's policy of dropping packets with the largest finish numbers maintains better fairness even during congestion, with drops distributed proportionally according to source priorities.

7.1.4 Server Utilization

Both FCFS and WFQ achieve similarly high server utilization rates when offered load exceeds link capacity. The scheduling algorithm choice has minimal impact on link efficiency, with both approaches effectively keeping the output link busy during periods of available traffic.

7.2 Key Insights

1. **Fairness Requires Mechanism:** Simple temporal ordering is insufficient for fair resource allocation in heterogeneous environments. Explicit fairness mechanisms like WFQ are necessary to ensure equitable service.

2. **Complexity Justification:** WFQ's increased computational complexity is justified by its superior fairness, isolation, and quality of service properties, particularly in multi-user network environments.

3. **Context-Dependent Optimality:** No single algorithm is universally optimal. FCFS suffices in homogeneous, low-load, or trusted environments, while WFQ is essential for heterogeneous, congested, or untrusted scenarios.

4. **Weight Configuration Importance:** In WFQ, appropriate weight assignment is critical for achieving desired service differentiation and meeting application requirements.

5. **Buffer Management Impact:** The buffer management strategy significantly influences fairness and drop characteristics, with WFQ's finish-number-based dropping providing superior fairness compared to FCFS drop-tail.